

06 — Cheatsheet commandes (adaptées au projet)

Se placer à la racine : `cd /c/wamp/www/renova-systems-projet-annuel` Conteneurs : `uc_mysql` , `uc_backend` ,
`uc_frontend` . DB : `pa2026` / user `upcycle` / mdp `upcyclePass123` (local) — sur la VM : user `upcycle` / mdp `upcycle` .

Lancer le projet

```
# Tout le projet (build + démarrage) en local
docker compose up -d --build

# Frontend local sur un autre port (ex 8088)
WEB_PORT=8088 docker compose up -d --build frontend

# Backend seul en Go SANS Docker (WAMP + MySQL local)
cd API && go run .
# (écoute sur :8081 ; le front WAMP tape localhost:8081)
```

Reconstruire / redémarrer

```
docker compose up -d --build backend      # rebuild + redémarre l'API seule
docker compose up -d --build frontend     # rebuild + redémarre le front seul
docker compose restart backend            # redémarre sans rebuild
docker compose down                       # arrête tout
docker compose down -v                    # arrête + SUPPRIME les volumes (reset DB !) ⚠
```

Logs

```
docker compose logs -f backend            # logs API en direct
docker compose logs -f frontend           # logs Nginx
docker compose logs --tail=100 mysql      # 100 dernières lignes MySQL
docker ps                                 # état des conteneurs + ports
```

Base de données

```
# Ouvrir un shell MySQL (local)
docker exec -it uc_mysql mysql -uupcycle -pupcyclePass123 pa2026

# Requête rapide
docker exec uc_mysql mysql -uupcycle -pupcyclePass123 pa2026 -e "SHOW TABLES;"

# EXPORTER la base (dump)
docker exec uc_mysql mysqldump -uupcycle -pupcyclePass123 pa2026 > sauvegarde_pa2026.sql

# IMPORTER un dump (⚠ utf8mb4 pour garder les accents)
```

```
docker exec -i uc_mysql mysql --default-character-set=utf8mb4 -uupcycle -pupcyclePass123 pa2026 < db/init.sql

# Reset complet de la base (réimport init.sql au prochain up)
docker compose down -v && docker compose up -d
```

Vérifs rapides (santé)

```
# L'API répond ? (401 = vivante, protégée)
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8081/admin/users

# Le front répond ?
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8088/

# Ports occupés (Windows / Git Bash)
netstat -ano | grep ':8081'
netstat -ano | grep ':8088'

# Variables d'env vues par le backend
docker exec uc_backend env | grep -E "DB_|STRIPE|ONESIGNAL|UPLOAD"
```

Tests

```
# Pas de tests Go automatisés dans le projet (à confirmer)
cd API && go build ./...      # au minimum : vérifie que ça compile
cd API && go vet ./...        # analyse statique
node -c Frontend/script/xxx.js # vérifie la syntaxe d'un fichier JS
```

Git (sauvegarde soutenance)

```
git status
git add -A
git commit -m "Sauvegarde avant soutenance"

# Branche de secours pour les modifs en direct
git checkout -b soutenance-modifs
# ... modifs ...
git checkout amel          # revenir à la branche stable si besoin
```

Docker Hub (mise à jour prod)

```
docker build -t amelbdj/upcycle-backend:vNN ./API
docker build -t amelbdj/upcycle-frontend:vNN ./Frontend
docker push amelbdj/upcycle-backend:vNN
docker push amelbdj/upcycle-frontend:vNN
# Sur la VM : éditer docker-compose-prod.yml (tags) puis :
docker compose pull && docker compose up -d
```